

CS-GY 9223 I: Lecture 3

Sketching, the Johnson-Lindenstrauss lemma + applications

NYU Tandon School of Engineering, Prof. Christopher Musco

Abstract architecture of a streaming algorithm:

- Given a dataset $D = d_1, \dots, d_n$ with n pieces of data, we want to output $f(D)$ for some function f .
- Maintain state S_t with $\ll |D|$ space at each time step t .
- **Update phase:** Receive $d_1, \dots, d_n$ in sequence, update $S_t \leftarrow U(S_{t-1}, d_t)$.
- **Process phase:** Using S_n , compute approximation to $f(D)$.

Typical setup for training models in machine learning, required for large scale data monitoring (e.g. processing sensor data, time series, seismic data, satellite imagery, etc.)



DISTINCT ELEMENTS PROBLEM

Input: $d_1, \dots, d_n \in \mathcal{U}$ where $\mathcal{U}$ is a huge universe of items.

Output: Number of distinct inputs, D .

Example: $f(1, 10, 10, 4, 9, 1, 1, 4) \rightarrow 4$

DISTINCT ELEMENTS PROBLEM

Input: $d_1, \dots, d_n \in \mathcal{U}$ where $\mathcal{U}$ is a huge universe of items.

Output: Number of distinct inputs, D .

Example: $f(1, 10, 10, 4, 9, 1, 1, 4) \rightarrow 4$

Naive solution takes $O(D)$ space and is exact. We want something that uses much less space, but is approximate.

DISTINCT ELEMENTS PROBLEM

Input: $d_1, \dots, d_n \in \mathcal{U}$ where $\mathcal{U}$ is a huge universe of items.

Output: Number of distinct inputs, D .

Example: $f(1, 10, 10, 4, 9, 1, 1, 4) \rightarrow 4$

Naive solution takes $O(D)$ space and is exact. We want something that uses much less space, but is approximate.

In practice: Approximate COUNT(DISTINCT) in huge databases (of weblogs, biological data, etc.).

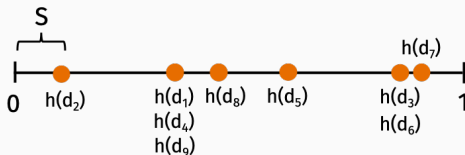
DISTINCT ELEMENTS PROBLEM

Input: $d_1, \dots, d_n \in \mathcal{U}$ where $\mathcal{U}$ is a huge universe of items.

Output: Number of distinct inputs.

Example: $f(1, 10, 10, 4, 9, 1, 1, 4) \rightarrow 4$

Basic estimator:

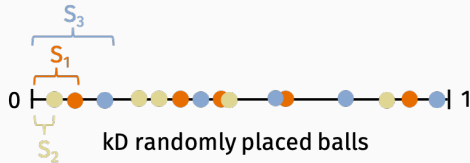


D randomly placed balls

$$\mathbb{E}[S] = \frac{1}{D+1}. \text{ Estimate } D \approx \frac{1}{S} - 1.$$

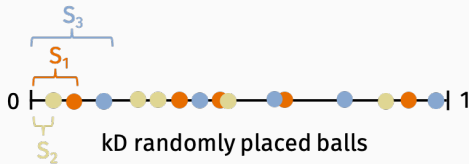
REPEAT TO BOOST ACCURACY

- Choose k random hash function $h_1, \dots, h_k : \mathcal{U} \rightarrow [0, 1]$.
- Maintain k estimators $S_1, \dots, S_k$.



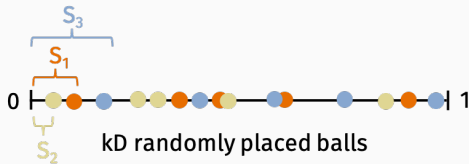
REPEAT TO BOOST ACCURACY

- Choose k random hash function $h_1, \dots, h_k : \mathcal{U} \rightarrow [0, 1]$.
- Maintain k estimators $S_1, \dots, S_k$.
- **Set:** $S' = \frac{1}{k} \sum_{i=1}^k S_k$.
- **Estimate:** $D \approx \frac{1}{S'} - 1$.



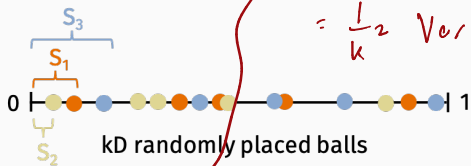
REPEAT TO BOOST ACCURACY

- Choose k random hash function $h_1, \dots, h_k : \mathcal{U} \rightarrow [0, 1]$.
- Maintain k estimators $S_1, \dots, S_k$.
- **Set:** $S' = \frac{1}{k} \sum_{i=1}^k S_k$.
- **Estimate:** $D \approx \frac{1}{S'} - 1$.



REPEAT TO BOOST ACCURACY

- Choose k random hash function $h_1, \dots, h_k : \mathcal{U} \rightarrow [0, 1]$.
- Maintain k estimators $S_1, \dots, S_k$.
- **Set:** $S' = \frac{1}{k} \sum_{i=1}^k S_i$
- **Estimate:** $D \approx \frac{1}{S'} - 1$.



$$\text{Var} \left[\frac{1}{n} \sum_{i=1}^n S_i \right]$$

$$= \frac{1}{k^2} \text{Var} \left[\sum_{i=1}^k S_i \right]$$

$$= \frac{1}{k^2} \sum_{i=1}^k \text{Var}[S_i]$$

$$= \frac{1}{k^2} \cdot k \cdot \sigma^2$$

$$= \frac{1}{k} \sigma^2$$

If $\text{Var}[S] = \sigma^2$, $\text{Var}[S'] = ?$

$\frac{1}{k} \sum_{i=1}^k S_i \rightarrow$ each S_i has variance σ^2 .

Applying Chebyshev's inequality: Need $O(1/\epsilon^2)$ estimators to return $\tilde{D}$ satisfying:

$$(1 - \epsilon)\tilde{D} \leq \tilde{D} \leq (1 + \epsilon)\tilde{D}$$

with probability 9/10.

In practice, we cannot hash to real numbers on $[0, 1]$. Instead, map to bit vectors.

DISTINCT ELEMENTS IN PRACTICE

In practice, we cannot hash to real numbers on $[0, 1]$. Instead, map to bit vectors.

purposely
Real Flajolet-Martin / HyperLogLog:

$h(x_1)$	1010010
$h(x_2)$	1001100
$h(x_3)$	1001110
⋮	
$h(x_n)$	1011000

DISTINCT ELEMENTS IN PRACTICE

In practice, we cannot hash to real numbers on $[0, 1]$. Instead, map to bit vectors.

Real Flajolet-Martin / HyperLogLog:

$h(x_1)$	10100 10
$h(x_2)$	10011 00
$h(x_3)$	10011 10
⋮	
$h(x_n)$	1011 000

- Estimate # distinct elements based on maximum number of trailing zeros m .
- The more distinct hashes we see, the higher we expect this maximum to be.

Flajolet-Durand / HyperLogLog:

$h(x_1)$	1010010
$h(x_2)$	1001100
$h(x_3)$	1001110
⋮	
$h(x_n)$	1011000

- Estimate # distinct elements based on maximum number of trailing zeros m .
- The more distinct hashes we see, the higher we expect this maximum to be.

With D distinct elements what do we expect m to be?

Flajolet-Durand / HyperLogLog:

$h(x_1)$	1010010
$h(x_2)$	1001100
$h(x_3)$	1001110
⋮	
$h(x_n)$	1011000

- Estimate # distinct elements based on maximum number of trailing zeros m .
- The more distinct hashes we see, the higher we expect this maximum to be.

With D distinct elements what do we expect m to be?

$$\Pr(h(x_i) \text{ has } x \text{ trailing zeros}) =$$

Flajolet-Durand / HyperLogLog:

$h(x_1)$	1010010
$h(x_2)$	1001100
$h(x_3)$	1001110
⋮	
$h(x_n)$	1011000

- Estimate # distinct elements based on maximum number of trailing zeros m .
- The more distinct hashes we see, the higher we expect this maximum to be.

With D distinct elements what do we expect m to be?

$$\Pr(h(x_i) \text{ has } x \text{ trailing zeros}) = \frac{1}{2^x}$$

Flajolet-Durand / HyperLogLog:

$h(x_1)$	1010010
$h(x_2)$	1001100
$h(x_3)$	1001110
⋮	
$h(x_n)$	1011000

- Estimate # distinct elements based on maximum number of trailing zeros m .
- The more distinct hashes we see, the higher we expect this maximum to be.

With D distinct elements what do we expect m to be?

$$\Pr(h(x_i) \text{ has } x \text{ trailing zeros}) = \frac{1}{2^x}$$

Flajolet-Durand / HyperLogLog:

$h(x_1)$	1010010
$h(x_2)$	1001100
$h(x_3)$	1001110
⋮	
$h(x_n)$	1011000

- Estimate # distinct elements based on maximum number of trailing zeros m .
- The more distinct hashes we see, the higher we expect this maximum to be.

With D distinct elements what do we expect m to be?

$$\Pr(h(x_i) \text{ has } \log D \text{ trailing zeros}) = \frac{1}{2^{\log D}}$$

Flajolet-Durand / HyperLogLog:

$h(x_1)$	1010010
$h(x_2)$	1001100
$h(x_3)$	1001110
⋮	
$h(x_n)$	1011000

- Estimate # distinct elements based on maximum number of trailing zeros m .
- The more distinct hashes we see, the higher we expect this maximum to be.

With D distinct elements what do we expect m to be?

$$\Pr(h(x_i) \text{ has } \log D \text{ trailing zeros}) = \frac{1}{2^{\log D}} = \frac{1}{D}.$$

Flajolet-Durand / HyperLogLog:

$h(x_1)$	1010010
$h(x_2)$	1001100
$h(x_3)$	1001110
⋮	
$h(x_n)$	1011000

- Estimate # distinct elements based on maximum number of trailing zeros m .
- The more distinct hashes we see, the higher we expect this maximum to be.

With D distinct elements what do we expect m to be?

$$\Pr(h(x_i) \text{ has } \log D \text{ trailing zeros}) = \frac{1}{2^{\log D}} = \frac{1}{D}.$$

So with D distinct hashes, expect to see 1 with $\log D$ trailing zeros. Expect $m \approx \log D$.

Flajolet-Durand / HyperLogLog:

$h(x_1)$	1010010
$h(x_2)$	1001100
$h(x_3)$	1001110
⋮	
$h(x_n)$	1011000

- Estimate # distinct elements based on maximum number of trailing zeros m .
- The more distinct hashes we see, the higher we expect this maximum to be.

With D distinct elements what do we expect m to be?

$$\Pr(h(x_i) \text{ has } \log D \text{ trailing zeros}) = \frac{1}{2^{\log D}} = \frac{1}{D}.$$

So with D distinct hashes, expect to see 1 with $\log D$ trailing zeros. Expect $m \approx \log D$. m takes $O(\log \log D)$ bits to store.

Total Space: $O\left(\frac{\log \log D}{\epsilon^2} + \log D\right)$ for an ϵ approximate count.

Total Space: $O\left(\frac{\log \log D}{\epsilon^2} + \log D\right)$ for an ϵ approximate count.

“Using an auxiliary memory smaller than the size of this abstract, the LogLog algorithm makes it possible to estimate in a single pass and within a few percents the number of different words in the whole of Shakespeare’s works.” – Flajolet, Durand.

LOGLOG SPACE

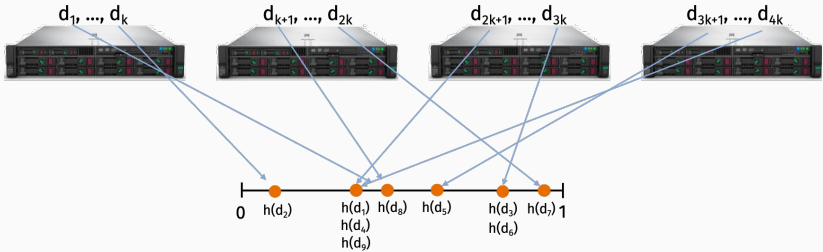
Total Space: $O\left(\frac{\log \log D}{\epsilon^2} + \log D\right)$ for an ϵ approximate count.

“Using an auxiliary memory smaller than the size of this abstract, the LogLog algorithm makes it possible to estimate in a single pass and within a few percents the number of different words in the whole of Shakespeare’s works.” – Flajolet, Durand.

Using HyperLogLog to count 1 billion distinct items with 2% accuracy:

$$\begin{aligned}\text{space used} &= O\left(\frac{\log \log D}{\epsilon^2} + \log D\right) \\ &= \frac{1.04 \cdot \lceil \log_2 \log_2 D \rceil}{\epsilon^2} + \lceil \log_2 D \rceil \text{ bits} \\ &= \frac{1.04 \cdot 5}{.02^2} + 30 = 13030 \text{ bits} \approx \underline{\underline{1.6 \text{ kB}}}\end{aligned}$$

DISTRIBUTED DISTINCT ELEMENTS



Implementations: Google PowerDrill, Facebook Presto, Twitter Algebird, Amazon Redshift.

Use Case: Exploratory SQL-like queries on tables with 100's of billions of rows.

- **Count** number of **distinct** users in Germany that made at least one search containing the word 'auto' in the last month.
- **Count** number of **distinct** subject lines in emails sent by users that have registered in the last week, in comparison to number of emails sent overall (to estimate rates of spam accounts).

HYPERLOGLOG IN PRACTICE

Implementations: Google PowerDrill, Facebook Presto, Twitter Algebird, Amazon Redshift.

Use Case: Exploratory SQL-like queries on tables with 100's of billions of rows.

- **Count** number of **distinct** users in Germany that made at least one search containing the word 'auto' in the last month.
- **Count** number of **distinct** subject lines in emails sent by users that have registered in the last week, in comparison to number of emails sent overall (to estimate rates of spam accounts).

Answering a query requires a (distributed) linear scan over the database: 2 seconds in Google's distributed implementation.

“The system has been in production since end of 2008 and was made available for internal users across all of Google mid 2009. Each month it is used by more than 800 users sending out about 4 million SQL queries. **After a hard day's work, one of our top users has spent over 6 hours in the UI, triggering up to 12 thousand queries.** When using our column-store as a backend, this may amount to scanning as much as 525 trillion cells in (hypothetical) full scans.”

Abstract architecture of a sketching algorithm:

- Given a dataset $D = d_1, \dots, d_n$ with n pieces of data, we want to output $f(D)$ for some function f .

Abstract architecture of a sketching algorithm:

- Given a dataset $D = d_1, \dots, d_n$ with n pieces of data, we want to output $f(D)$ for some function f .
- **Sketch phase:** For each $i \in 1, \dots, n$, compute $s_i = C(d_i)$, where C is some compression function and $|s_i| \ll d_i$.

Abstract architecture of a sketching algorithm:

- Given a dataset $D = d_1, \dots, d_n$ with n pieces of data, we want to output $f(D)$ for some function f .
- **Sketch phase:** For each $i \in 1, \dots, n$, compute $s_i = C(d_i)$, where C is some compression function and $|s_i| \ll d_i$.
- **Process phase:** Using (lower dimensional) dataset $s_1, \dots, s_n$, compute an approximation to $f(D)$.

Abstract architecture of a sketching algorithm:

- Given a dataset $D = d_1, \dots, d_n$ with n pieces of data, we want to output $f(D)$ for some function f .
- **Sketch phase:** For each $i \in 1, \dots, n$, compute $s_i = C(d_i)$, where C is some compression function and $|s_i| \ll d_i$.
- **Process phase:** Using (lower dimensional) dataset $s_1, \dots, s_n$, compute an approximation to $f(D)$.

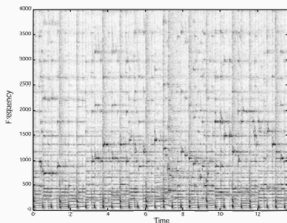


Sketching phase is easily distributed, parallelized, etc. Better space complexity, communication complexity, runtime, all at once.

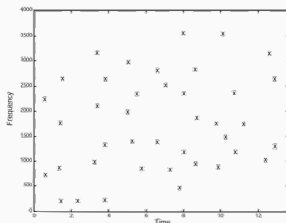
How does **Shazam** match a song clip against a library of 8 million songs (32 TB of data) in a fraction of a second?

SIMILARITY ESTIMATION

How does **Shazam** match a song clip against a library of 8 million songs (32 TB of data) in a fraction of a second?



Spectrogram extracted from audio clip.



Processed spectrogram: used to construct audio “fingerprint” $\mathbf{q} \in \{0, 1\}^d$.

Each clip is represented by a high dimensional binary vector $\mathbf{q}$.

1	0	1	1	0	0	0	1	0	0	0	0	1	1	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Given $\mathbf{q}$, find any nearby “fingerprint” $\mathbf{y}$ in a database – i.e. any $\mathbf{y}$ with $\text{dist}(\mathbf{y}, \mathbf{q})$ small.

Given $\mathbf{q}$, find any nearby “fingerprint” $\mathbf{y}$ in a database – i.e. any $\mathbf{y}$ with $\text{dist}(\mathbf{y}, \mathbf{q})$ small.

Challenges:

- Database is possibly huge: $O(nd)$ bits.
- Expensive to compute $\text{dist}(\mathbf{y}, \mathbf{q})$: $O(d)$ time.

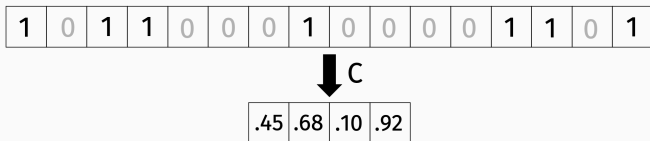
Goal: Design a more compact sketch for comparing $\mathbf{q}, \mathbf{y} \in \{0, 1\}^d$. Ideally $\ll d$ space/time complexity.

SIMILARITY ESTIMATION

Goal: Design a more compact sketch for comparing $\mathbf{q}, \mathbf{y} \in \{0, 1\}^d$. Ideally $\ll d$ space/time complexity.

$$C(\mathbf{q}) \in \mathbb{R}^k$$

$$C(\mathbf{y}) \in \mathbb{R}^k$$

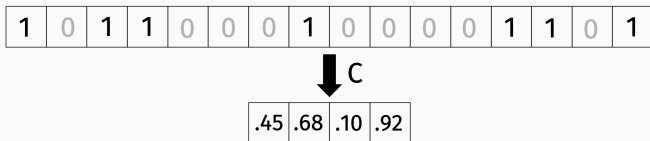


SIMILARITY ESTIMATION

Goal: Design a more compact sketch for comparing $\mathbf{q}, \mathbf{y} \in \{0, 1\}^d$. Ideally $\ll d$ space/time complexity.

$$C(\mathbf{q}) \in \mathbb{R}^k$$

$$C(\mathbf{y}) \in \mathbb{R}^k$$



Homomorphic Compression:

$C(\mathbf{q})$ should be similar to $C(\mathbf{y})$ if $\mathbf{q}$ is similar to $\mathbf{y}$.

Definition (Jaccard Similarity)

$$J(\mathbf{q}, \mathbf{y}) = \frac{|\mathbf{q} \cap \mathbf{y}|}{|\mathbf{q} \cup \mathbf{y}|} = \frac{\text{\# of non-zero entries in common}}{\text{total \# of non-zero entries}}$$

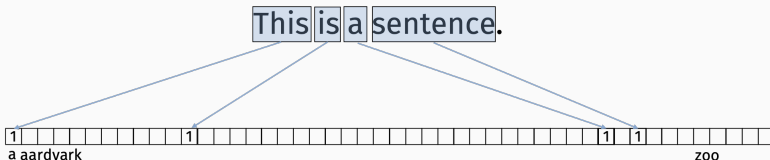
Natural similarity measure for binary vectors. $0 \leq J(\mathbf{q}, \mathbf{y}) \leq 1$.

Other applications:

- Change detection in documents (high speed web caches).
- Analyzing seismic data (matching signatures of earthquakes).
- User recommendations on social networking sites.

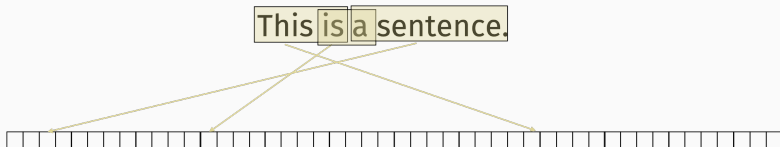
JACCARD SIMILARITY FOR DOCUMENT COMPARISON

“Bag-of-words” model:



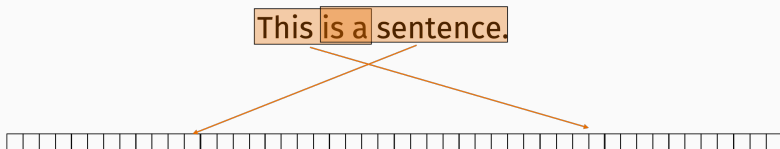
How many **words** do a pair of documents have in common?

“Bag-of-words” model:



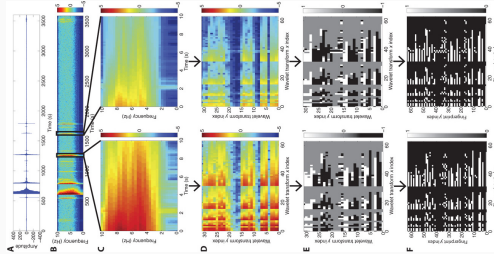
How many **bigrams** do a pair of documents have in common?

“Bag-of-words” model:



How many **trigrams** do a pair of documents have in common?

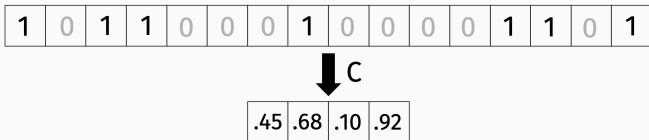
JACCARD SIMILARITY FOR SEISMIC DATA



Feature extract pipeline for earthquake data.

SIMILARITY ESTIMATION

Goal: Design a compact sketch $C : \{0, 1\} \rightarrow \mathbb{R}^k$:



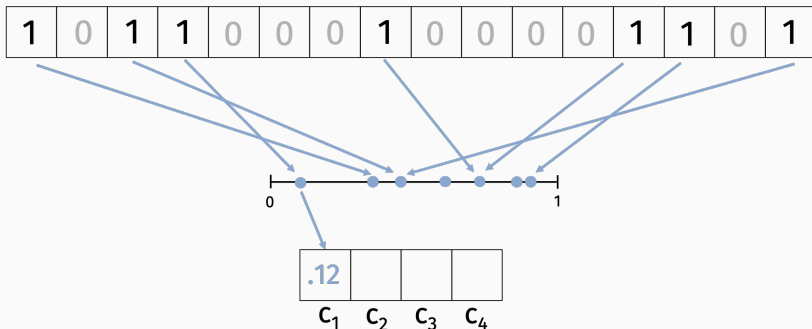
Homomorphic Compression: Want to use $C(\mathbf{q})$, $C(\mathbf{y})$ to approximately compute the Jaccard similarity $J(\mathbf{q}, \mathbf{y})$.

MinHash (Broder, '97):

- Choose k random hash functions
 $h_1, \dots, h_k : \{1, \dots, n\} \rightarrow [0, 1]$.
- For $i \in 1, \dots, k$, let $c_i = \min_{j, q_j=1} h_i(j)$.
- $C(\mathbf{q}) = [c_1, \dots, c_k]$.

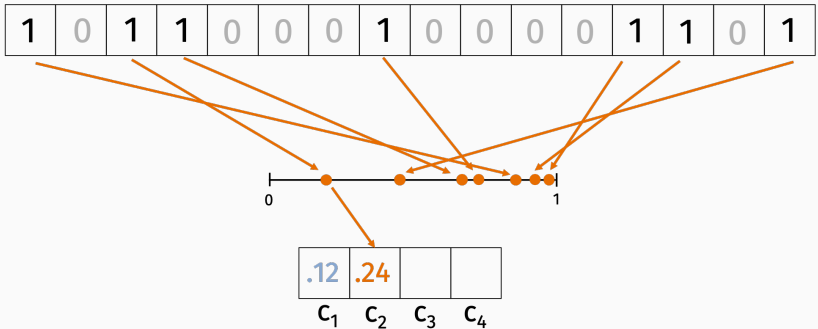
MinHash (Broder, '97):

- Choose k random hash functions
 $h_1, \dots, h_k : \{1, \dots, n\} \rightarrow [0, 1]$.
- For $i \in 1, \dots, k$, let $c_i = \min_{j, q_j=1} h_i(j)$.
- $C(\mathbf{q}) = [c_1, \dots, c_k]$.



MINHASH

- Choose k random hash functions
 $h_1, \dots, h_k : \{1, \dots, n\} \rightarrow [0, 1]$.
- For $i \in 1, \dots, k$, let $c_i = \min_{j, q_j=1} h_i(j)$.
- $C(q) = [c_1, \dots, c_k]$.



MINHASH ANALYSIS

Claim: $\Pr[c_i(q) = c_i(y)] = J(q, y)$.



$L(y)$

.1	.5	.2	.6
----	----	----	----

q

$L(q)$



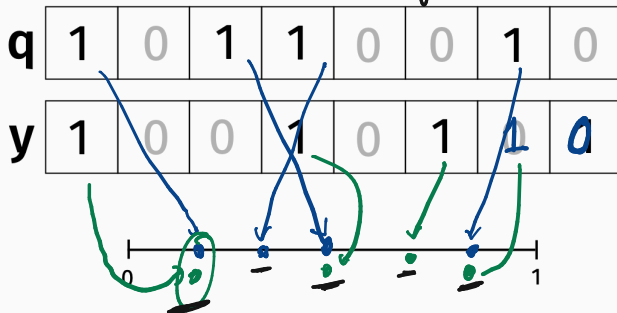
1	1	.5	.4	0
---	---	----	----	---

$$J(q, y) = 1$$

$$\Pr[c_i(q) = c_i(y)] = 1$$

MINHASH ANALYSIS

Claim: $\Pr[c_i(q) = c_i(y)] = J(q, y) \approx \frac{q \cap y}{q \cup y}$



Total # of distinct real values: $q \cup y$

total # of "repeat" positions: $q \cap y$

MINHASH ANALYSIS

Return $\tilde{J} = \frac{1}{k} \sum_{i=1}^k \mathbb{1}[c_i(q) = c_i(y)]$.

$i=1$
 $i=3$

Unbiased estimate for Jaccard similarity: $\mathbb{E}\tilde{J} = J(q, y)$.

$c(q)$	<u>.12</u>	.24	<u>.76</u>	.35	$c(y)$	<u>.12</u>	.98	<u>.76</u>	.11
	1	2	...	k					

MINHASH ANALYSIS

Chernoff bound: Analysis is the same as summing random coin flips. As long as $k = O\left(\frac{\log(1/\Delta)}{\epsilon^2}\right)$, then with prob $1 - \Delta$,

$$I(q, y) - \epsilon \leq \tilde{I}(C(q), C(y)) \leq I(q, y) + \epsilon$$

Pr this does not happen
 $\leq \delta$

Chernoff bound: Analysis is the same as summing random coin flips. As long as $k = O\left(\frac{\log(1/\delta)}{\epsilon^2}\right)$, then with prob. $1 - \delta$,

$$J(\mathbf{q}, \mathbf{y}) - \epsilon \leq \tilde{J}(C(\mathbf{q}), C(\mathbf{y})) \leq J(\mathbf{q}, \mathbf{y}) + \epsilon.$$

And $\tilde{J}$ only takes $O(k)$ time to compute! **Independent** of original fingerprint dimension d .

MINHASH ANALYSIS

$$\Pr[A \text{ or } B] \leq \underline{\underline{\Pr[A]}} + \underline{\underline{\Pr[B]}}$$

$$\tilde{J} = \frac{1}{k} \sum_{i=1}^k \mathbb{1}[c_i(q) = c_i(y)]$$

Suffices to prove:

- 1) $\tilde{J} \leq J + \epsilon$ with probability $(1 - \frac{\Delta}{2})$
- 2) $\tilde{J} \geq J - \epsilon$ with probability $(1 - \frac{\Delta}{2})$

$$\begin{aligned} &\xrightarrow{A} \left[\Pr[\tilde{J} > J + \epsilon] \leq \frac{\Delta}{2} \right. \\ &\quad \left. \Pr[\tilde{J} < J - \epsilon] \leq \frac{\Delta}{2} \right] \\ &\quad \xrightarrow{B} \end{aligned}$$

$\Pr[\text{neither } A \text{ nor } B] \text{ happens with prob } > 1 - \Delta$

$$\boxed{\Pr[A \text{ or } B] \leq \Delta}$$

MINHASH ANALYSIS

Theorem (Chernoff Bound, 1)

Let $X_1, X_2, \dots, X_k$ be independent $\{0, 1\}$ -valued random variables and let $p_i = \mathbb{E}[X_i]$, where $0 < p_i < 1$. Then the sum $X = \sum_{i=1}^k X_i$, which has mean $\mu = \sum_{i=1}^k p_i$, satisfies

Want to prove:

$$\Pr[\tilde{J} \geq J + \epsilon] \leq \frac{1}{2\Delta} \quad \text{if } k = O\left(\frac{\log(1/\Delta)}{\epsilon^2}\right) \quad \Pr[X \geq (1 + \delta)\mu] \leq e^{\frac{-\delta^2 \mu}{3 + 3\delta}}.$$

$$\tilde{J} = \frac{1}{k} \sum_{i=1}^k \underbrace{\mathbb{I}[L_i(q) = L_i(y)]}_{\substack{\{0,1\} \text{ random variable} \\ \text{that's one w/ prob. } J}} \longrightarrow \text{call this } X_i \text{ so, } \tilde{J} = \frac{1}{k} \sum_{i=1}^k X_i \quad \tilde{J} = \frac{1}{n} X.$$

$$1) \mu = \sum_{i=1}^k p_i = kJ$$

$$5) \Pr[\tilde{J} \geq J + \epsilon] \leq e^{-\epsilon^2 k / J} / 3 + 3\epsilon / 5.$$

$$2) \Pr[X \geq (1 + \delta)kJ] \leq e^{-\delta^2 kJ / 3 + 3\delta}$$

$$6) \Pr[\tilde{J} \geq J + \epsilon] \leq e^{-\epsilon^2 k / 3J + 3\epsilon} \leq e^{-\epsilon^2 k / 6} \quad \text{since } J, \epsilon < 1$$

$$3) \Pr[\tilde{J} \geq (1 + \delta)J] \leq e^{-\delta^2 kJ / 3 + 3\delta}$$

$$7) \text{ Set } k = \frac{12 \log(1/\Delta)}{\epsilon^2} = O\left(\frac{\log(1/\Delta)}{\epsilon^2}\right)$$

$$7) \text{ Set } \delta = \epsilon / J. \rightarrow \text{plug in}$$

$$\checkmark 8) \Pr[\tilde{J} \geq J + \epsilon] \leq e^{-2 \log(1/\Delta)} = \frac{1}{\Delta^2} \leq \frac{1}{2\Delta} \quad \text{for } \Delta \leq \frac{1}{2}$$

MINHASH ANALYSIS

Theorem (Chernoff Bound, 2)

Let $X_1, X_2, \dots, X_k$ be independent $\{0, 1\}$ -valued random variables and let $p_i = \mathbb{E}[X_i]$, where $0 < p_i < 1$. Then the sum $X = \sum_{i=1}^k X_i$, which has mean $\mu = \sum_{i=1}^k p_i$, satisfies

$$\Pr[X \leq (1 - \delta)\mu] \leq e^{-\frac{\delta^2 \mu}{3}}.$$

Want to prove:

$\Pr[\tilde{J} \leq J - \epsilon] \leq 1/2\Delta$ if $k = O\left(\frac{\log(1/\Delta)}{\epsilon^2}\right)$. NOTE This is trivially true when $J \leq \epsilon$! $\Pr[\tilde{J} \leq \text{negative \#}] = 0$ always positive.

for all $0 < \delta \leq 1$.

Some set up as previous page.

1) $\mu = kJ$.

2) $\Pr[X \leq (1 - \delta)kJ] \leq e^{-\delta^2 kJ/3}$

3) $\Pr[\tilde{J} \leq J - \delta J] \leq e^{-\delta^2 kJ/3}$

4) Set $\delta = \epsilon/J$. Theorem only applies when $\delta < 1$!

5) $\Pr[\tilde{J} \leq J - \epsilon J] \leq e^{-\epsilon^2 k/3J} \leq e^{-\epsilon^2 k/3}$

6) Set $k = \frac{12 \log(1/\Delta)}{\epsilon^2}$ as before.

SO only left to prove for $J > \epsilon$.

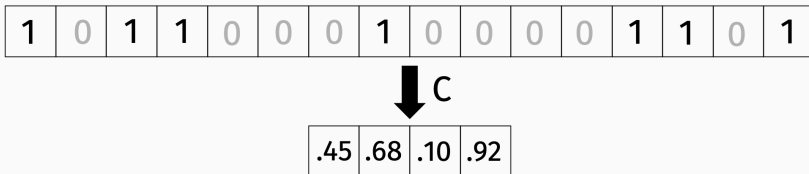
when $J > \epsilon$, $\epsilon/J < 1$, so we're okay!

7) $\Pr[\tilde{J} \leq J - \epsilon J] \leq e^{-\gamma \log(1/\Delta)}$
 $= 1/\Delta^\gamma$

8) $\Pr[\tilde{J} \leq J - \epsilon J] \leq 1/2\Delta$ for $\Delta < 1/2 \cdot 28$
 $\leq 1/2\Delta$

One incredibly powerful theorem:
The Johnson-Lindenstrauss Lemma.

EUCLIDEAN DIMENSIONALITY REDUCTION



EUCLIDEAN DIMENSIONALITY REDUCTION

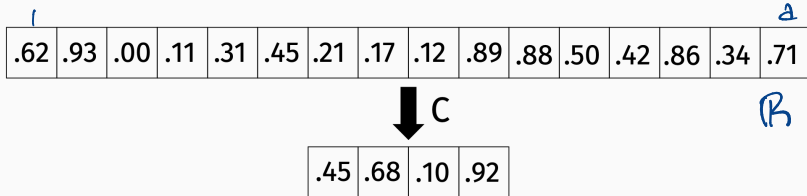
.62	.93	.00	.11	.31	.45	.21	.17	.12	.89	.88	.50	.42	.86	.34	.71
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----



C

.45	.68	.10	.92
-----	-----	-----	-----

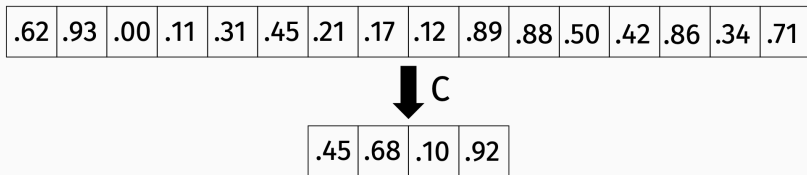
EUCLIDEAN DIMENSIONALITY REDUCTION



Euclidean norm / distance:

- Given $\mathbf{q} \in \mathbb{R}^d$, $\|\mathbf{q}\|_2 = \sqrt{\sum_{i=1}^d q(i)^2}$.
- Given $\mathbf{q}, \mathbf{y} \in \mathbb{R}^d$, distance defined as $\|\mathbf{q} - \mathbf{y}\|_2$.

EUCLIDEAN DIMENSIONALITY REDUCTION



Euclidean norm / distance:

- Given $\mathbf{q} \in \mathbb{R}^d$, $\|\mathbf{q}\|_2 = \sqrt{\sum_{i=1}^d q(i)^2}$.
- Given $\mathbf{q}, \mathbf{y} \in \mathbb{R}^d$, distance defined as $\|\mathbf{q} - \mathbf{y}\|_2$.

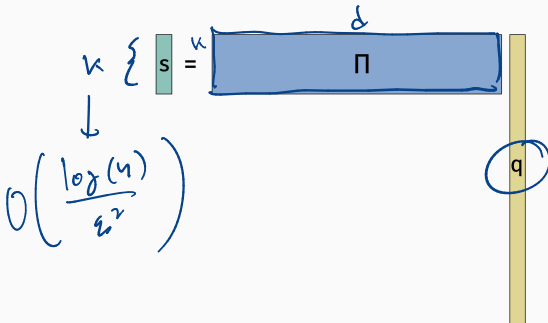
Can we find compact sketches that preserve Euclidean distance, just as we did for Jaccard similarity?

EUCLIDEAN DIMENSIONALITY REDUCTION

Lemma (Johnson-Lindenstrauss, 1984) JL

For any set of n data points $\mathbf{q}_1, \dots, \mathbf{q}_n \in \mathbb{R}^d$ there exists a linear map $\Pi : \mathbb{R}^d \rightarrow \mathbb{R}^k$ where $k = O\left(\frac{\log n}{\epsilon^2}\right)$ such that for all i, j ,

$$(1 - \epsilon) \|\mathbf{q}_i - \mathbf{q}_j\|_2 \leq \|\Pi \mathbf{q}_i - \Pi \mathbf{q}_j\|_2 \leq (1 + \epsilon) \|\mathbf{q}_i - \mathbf{q}_j\|_2.$$



EUCLIDEAN DIMENSIONALITY REDUCTION

Remarkably, Π can be chosen completely at random!

One possible construction: Random Gaussian.

$$\Pi_{ij} = \frac{1}{\sqrt{k}} \mathcal{N}(0, 1) \quad \frac{1}{\sqrt{n}} \left[\begin{array}{ccc} \circ & \circ & \circ \end{array} \right]$$

Remarkably, Π can be chosen completely at random!

One possible construction: Random Gaussian.

$$\Pi_{i,j} = \frac{1}{\sqrt{k}} \mathcal{N}(0, 1)$$

The map Π is **oblivious to the data set**. This stands in contrast to e.g. PCA, amongst other differences.

Remarkably, Π can be chosen completely at random!

One possible construction: Random Gaussian.

$$\Pi_{i,j} = \frac{1}{\sqrt{k}} \mathcal{N}(0, 1)$$

The map Π is **oblivious to the data set**. This stands in contrast to e.g. PCA, amongst other differences.

[Indyk, Motwani 1998] [Arriaga, Vempala 1999] [Achlioptas 2001]
[Dasgupta, Gupta 2003].

Many other possible choices suffice – you can use random $\{+1, -1\}$ variables, sparse random matrices, pseudorandom Π . Each with different advantages. We should have time to discuss a few examples next lecture.

Intermediate result: (which we already know how to prove)

Lemma (Distributional JL Lemma)

Let $\mathbf{\Pi} \in \mathbb{R}^{k \times d}$ be chosen so that each entry equals $\frac{1}{\sqrt{k}}\mathcal{N}(0, 1)$, where $\mathcal{N}(0, 1)$ denotes a standard Gaussian random variable.

If we choose $k = O\left(\frac{\log(1/\delta)}{\epsilon^2}\right)$, then for any vector $\mathbf{q}$, with probability $(1 - \delta)$:

$$(1 - \epsilon)\|\mathbf{q}\|_2 \leq \|\mathbf{\Pi}\mathbf{q}\|_2 \leq (1 + \epsilon)\|\mathbf{q}\|_2$$

EUCLIDEAN DIMENSIONALITY REDUCTION

Intermediate result: (which we already know how to prove)

Lemma (Distributional JL Lemma)

Let $\Pi \in \mathbb{R}^{k \times d}$ be chosen so that each entry equals $\frac{1}{\sqrt{k}} \mathcal{N}(0, 1)$, where $\mathcal{N}(0, 1)$ denotes a standard Gaussian random variable.

If we choose $k = O\left(\frac{\log(1/\delta)}{\epsilon^2}\right)$, then for any vector $\mathbf{q}$, with probability $(1 - \delta)$:

$$(1 - \epsilon) \|\mathbf{q}\|_2^2 \leq \|\Pi \mathbf{q}\|_2^2 \leq (1 + \epsilon) \|\mathbf{q}\|_2^2$$

$\nearrow q_i - q_j$ for all

$$\|\Pi(q_i - q_j)\|_2 = \|\Pi q_i - \Pi q_j\|_2$$

In class exercise: Given this lemma, prove the Johnson-Lindenstrauss lemma.

Implies: $\rightarrow$ If $k = O(\log n / \epsilon^2)$ then with prob. $1 - \delta$

$$(1 - \epsilon) \|q_i - q_j\|_2 \leq \|\Pi q_i - \Pi q_j\|_2 \leq (1 + \epsilon) \|q_i - q_j\|_2 \quad \text{for all } i, j \in \{1, \dots, n\}$$

IN CLASS EXERCISE

For ~~any~~ i, j if $k = O\left(\frac{\log(1/\delta)}{\epsilon^2}\right)$
 then w/ probability $(1-\delta)$ $\boxed{S_{ij}}$
 $(1-\epsilon) \|q_i - q_j\|_2 \leq \|\pi q_i - \pi q_j\|_2 \leq (1+\epsilon) \|q_i - q_j\|_2$

$q_1 \dots q_n$ $\binom{n}{2}$ i, j pairs $\rightarrow O(n^2)$

$\delta = \log\left(\frac{1}{10n^2}\right)$, $\Pr(S_{ij}) \geq 1 - \frac{1}{10n^2}$ for all i, j

$k = \frac{\log(10n^2)}{\epsilon^2} \rightarrow O(\log n)$ $\binom{n}{2}$ $\Pr(S_{i_1} \text{ and } S_{i_2} \text{ and } S_{i_3} \dots)$
 $\geq 1 - \frac{\binom{n}{2}}{10n^2}$
 $\geq 1 - \frac{1}{10} = \frac{9}{10}$

IN CLASS EXERCISE

Want to argue that with high probability, $\|\Pi \mathbf{q}\|_2 = (1 \pm \epsilon)\|\mathbf{q}\|_2$.
It suffices to prove that, with probability $(1 - \delta)$,

$$(1 - \epsilon)\|\mathbf{q}\|_2^2 \leq \|\Pi \mathbf{q}\|_2^2 \leq (1 + \epsilon)\|\mathbf{q}\|_2^2$$

Want to argue that with high probability, $\|\Pi \mathbf{q}\|_2 = (1 \pm \epsilon)\|\mathbf{q}\|_2$.
It suffices to prove that, with probability $(1 - \delta)$,

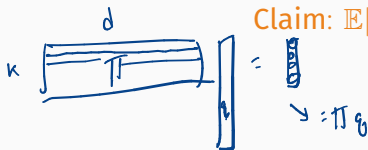
$$(1 - \epsilon)\|\mathbf{q}\|_2^2 \leq \|\Pi \mathbf{q}\|_2^2 \leq (1 + \epsilon)\|\mathbf{q}\|_2^2$$

Claim: $\mathbb{E}\|\Pi \mathbf{q}\|_2^2 = \|\mathbf{q}\|_2^2$.

PROOF OF DISTRIBUTIONAL JL

Want to argue that with high probability, $\|\Pi \mathbf{q}\|_2 = (1 \pm \epsilon)\|\mathbf{q}\|_2$.
It suffices to prove that, with probability $(1 - \delta)$,

$$(1 - \epsilon)\|\mathbf{q}\|_2^2 \leq \|\Pi \mathbf{q}\|_2^2 \leq (1 + \epsilon)\|\mathbf{q}\|_2^2$$



Claim: $\mathbb{E}\|\Pi \mathbf{q}\|_2^2 = \|\mathbf{q}\|_2^2$.

$$\mathbb{E}[\|\Pi \mathbf{q}\|_2^2] = \|\mathbf{q}\|_2^2$$

$$\mathbb{E}[Y^2] = k \mathbb{E}[(\Pi \mathbf{q})_i^2] = \|\mathbf{q}\|_2^2$$

$$\|\Pi \mathbf{q}\|_2^2 = \sum_{i=1}^k [\Pi \mathbf{q}]_i^2$$

$$\mathbb{E}\|\Pi \mathbf{q}\|_2^2 = \sum_{i=1}^k \mathbb{E}([\Pi \mathbf{q}]_i^2)$$

$$k \mathbb{E}[(\Pi \mathbf{q})_i^2]$$

$$= \|\mathbf{q}\|_2^2$$

PROOF OF DISTRIBUTIONAL JL

Goal: Let $Y = \sqrt{k} \cdot [\mathbf{pq}]_j$. Evaluate $\mathbb{E}[Y^2]$

$$Y = \sqrt{n} \sum_{j=1}^d \frac{X_j}{\sqrt{n}} \cdot g_j$$

$\left[\frac{1}{\sqrt{n}} X_1, \dots, \frac{1}{\sqrt{n}} X_d \right]$
 $\frac{1}{\sqrt{n}} N(0, 1)$

$X_1, \dots, X_d \sim N(0, 1)$

$$Y = \sum_{j=1}^d X_j \cdot g_j$$

$$\mathbb{E}[Y] = \sum_{j=1}^d g_j \mathbb{E}[X_j] \stackrel{\rightarrow 0}{=} 0$$

$$\text{Var}[Y] = \mathbb{E}[Y^2] - \mathbb{E}[Y]^2 \stackrel{\rightarrow 0}{=} \mathbb{E}[Y^2]$$

$$\begin{aligned} \text{Var}[Y] &= \sum_{j=1}^d \text{Var}[X_j \cdot g_j] \stackrel{\rightarrow 1}{=} \sum_{j=1}^d g_j^2 \text{Var}[X_j] \\ &= \sum_{j=1}^d g_j^2 = \|\mathbf{g}\|_2^2 \end{aligned}$$



What type of random variable is $\boxed{[\Pi q]_i^2}$? $= \chi^2$

What type of random variable is $[\Pi q]_i^2$?

Fact (Stability of Gaussian random variables)

$$\mathcal{N}(\mu_1, \sigma_1^2) + \mathcal{N}(\mu_2, \sigma_2^2) = \mathcal{N}(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2)$$

o(i). $\sum_{i=1}^d g_i x_i \rightarrow \mathcal{N}(0, 1)$

$\mathcal{N}(0, \|g\|_2^2)$

What type of random variable is $[\mathbf{q}]_i^2$?

Fact (Stability of Gaussian random variables)

$$\mathcal{N}(\mu_1, \sigma_1^2) + \mathcal{N}(\mu_2, \sigma_2^2) = \mathcal{N}(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2)$$

$[\mathbf{q}]_i^2$ is the square of a Gaussian random variable and $\|\mathbf{x}\|_2^2$ is a sum of k squared Gaussian random variables.

What type of random variable is $[\mathbf{q}]_i^2$?

Fact (Stability of Gaussian random variables)

$$\mathcal{N}(\mu_1, \sigma_1^2) + \mathcal{N}(\mu_2, \sigma_2^2) = \mathcal{N}(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2)$$

$[\mathbf{q}]_i^2$ is the square of a Gaussian random variable and $\|\mathbf{q}\|_2^2$ is a sum of k squared Gaussian random variables.

“Chi-squared random variable with k degrees of freedom.”

Lemma

Let X be a chi-squared random variable with k degrees of freedom.

$$\Pr[|\mathbb{E}X - X| \geq \epsilon \mathbb{E}X] \leq 2e^{-k\epsilon^2/8}$$

SAMPLE APPLICATION

k-means clustering: Give data points $\mathbf{x}_1, \dots, \mathbf{x}_n$, find centers $\mu_1, \dots, \mu_k$ to minimize:

$$\text{Cost}(\mu_1, \dots, \mu_k) = \sum_{i=1}^{\overbrace{n}} \min_{j=1, \dots, k} \|\underline{\mu_j} - \underline{\mathbf{x}_i}\|_2^2$$

k



K-MEANS CLUSTERING

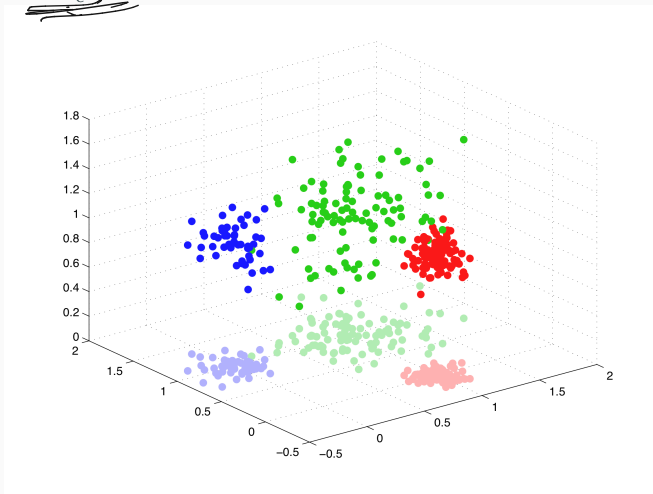
Equivalent formulation: Find clusters $C_1, \dots, C_k \subseteq \{1, \dots, n\}$ to minimize:



$$\text{Cost}(\underline{C_1}, \dots, C_k) = \sum_{j=1}^k \frac{1}{2|C_j|} \sum_{u,v \in C_j} \underline{\underline{\|x_u - x_v\|_2^2}}.$$

K-MEANS CLUSTERING

Approximation algorithm: Find optimal clusters $\tilde{C}_1, \dots, \tilde{C}_k$ for the $k = O(\frac{\log n}{\epsilon^2})$ dimension data set $\mathbf{X}_1, \dots, \mathbf{X}_n$.



K-MEANS CLUSTERING

$$\text{Cost}(C_1, \dots, C_k) = \sum_{j=1}^k \frac{1}{2|C_j|} \sum_{u,v \in C_j} \|x_u - x_v\|_2^2.$$

$$\widetilde{\text{Cost}}(C_1, \dots, C_k) = \sum_{j=1}^k \frac{1}{2|C_j|} \sum_{u,v \in C_j} \underbrace{\| \Pi x_u - \Pi x_v \|_2^2}_{(1-\epsilon)\|x_u - x_v\|_2^2 \leq \dots \leq (1+\epsilon)\|x_u - x_v\|_2^2}$$

$$(1-\epsilon)\|x_u - x_v\|_2^2 \leq \dots \leq (1+\epsilon)\|x_u - x_v\|_2^2$$

$$(1-\epsilon) \text{Cost}(C_1, \dots, C_k) \leq \widetilde{\text{Cost}}(C_1, \dots, C_k) \stackrel{(1+\epsilon)}{\leq} \text{Cost}(C_1, \dots, C_k)$$